BLOOD BANK GIT CONTRIB HANDOUT

No-Conflict Branching Strategy: 1 Developer = 1 File (120 Commits)

Rules: Each member edits exactly ONE file to prevent any merge conflicts during merges.

DAGIM (dagi931)

Exclusive File: **`backend/src/controllers/authController.js`**

Feature Branch: *`git checkout -b dagim/reset-endpoints`*

Commit: 1. init: add sendForgotPasswordEmail helper skeleton

```
async function sendForgotPasswordEmail(toEmail, code) {
  // Config settings
}
```

Commit: 2. email: setup sendForgotPasswordEmail headers and body properties

```
headers: { 'accept': 'application/json', 'api-key': apiKey }
```

Commit: 3. email: embed html content with password recovery code

```
htmlContent: `<h3>Use code: ${code} to reset password</h3>`
```

Commit: 4. email: log success result from brevo smtp gateway API

```
console.log(`Password reset code ${code} sent to ${toEmail}`);
```

Commit: 5. feat: add forgotPassword controller request handler function

```
async function forgotPassword(req, res) {
  const { email } = req.body;
}
```

Commit: 6. feat: lookup user credentials in database using email

```
const user = await mainDb.user.findUnique({ where: { email } });
```

Commit: 7. feat: reject forgotPassword requests if email is not found

```
if (!user) return res.status(400).json({ error: 'No account found' });
```

Commit: 8. feat: check user role to select correct reset mechanism

```
if (user.role === 'donor') { /* OTP flow */ }
```

Commit: 9. feat: generate secure 6-digit verification numeric code

```
const code = Math.floor(100000 + Math.random() * 900000).toString();
```

Commit: 10. db: upsert reset code in emailVerification table with expiry

```
await mainDb.emailVerification.upsert({ where: { email }, update: { code } });
```

Commit: 11. feat: dispatch donor password reset recovery email with code

```
await sendForgotPasswordEmail(email, code);
```

Commit: 12. feat: check for duplicate workstation resets under pending status

```
const ticket = await mainDb.passwordResetRequest.findFirst({ where: { user_id } });
```

Commit: 13. db: create new passwordResetRequest record for workstation staff

```
await mainDb.passwordResetRequest.create({ data: { user_id, email } });
```

Commit: 14. feat: add resetPasswordDonor handler for code validation

```
async function resetPasswordDonor(req, res) {
  const { email, code } = req.body;
}
```

Commit: 15. feat: verify reset code exists in emailVerification database

```
const record = await mainDb.emailVerification.findUnique({ where: { email } });
```

Commit: 16. feat: validate user verification code matches stored credentials

```
if (record.code !== code) return res.status(400).json({ error: 'Invalid' });
```

Commit: 17. feat: validate timestamp does not exceed verification expiry

```
if (new Date() > record.expires_at) return res.status(400).json({ error: 'Expired' });
```

Commit: 18. auth: encrypt new secure password using bcrypt hashing

```
const passwordHash = await bcrypt.hash(newPassword, 10);
```

Commit: 19. db: update user record with new passwordHash value

```
await mainDb.user.update({ where: { email }, data: { password_hash: passwordHash } });
```

Commit: 20. db: delete temporary emailVerification record on success

```
await mainDb.emailVerification.delete({ where: { email } });
```

BEKAM (bekamdawit333)

Exclusive File: **`backend/prisma/schema.prisma`**

Feature Branch: *`git checkout -b bekam/hms-models`*

Commit: 1. init: define postgresql datasource provider database configuration

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

Commit: 2. model: define base User model fields and map users table

```
model User {
  id    String @id @default(uuid())
  email String @unique
}
```

Commit: 3. model: add status and role fields to User model definition

```
  role   String
  status String @default("pending")
```

Commit: 4. model: define base Donor model profile metadata fields

```
model Donor {
  fayda_id String @id
  name     String
}
```

Commit: 5. model: add user_id foreign key relation to Donor model

```
  user_id String? @unique
  user    User?   @relation(fields: [user_id], references: [id])
```

Commit: 6. model: define BloodSample model fields for testing logs

```
model BloodSample {
  id         String @id @default(uuid())
  blood_type String
}
```

Commit: 7. model: add patient_id relation link to BloodSample model

```
  patient_id String?
  patient    Patient? @relation(fields: [patient_id], references: [id])
```

Commit: 8. model: define WarehouseStock model schema with unique constraints

```
model WarehouseStock {
  id         String @id @default(uuid())
  blood_type String @unique
}
```

Commit: 9. model: define HospitalRequest model with default pending status

```
model HospitalRequest {
  id     String @id @default(uuid())
  status String @default("pending")
}
```

Commit: 10. model: define SentSmsLog database schema configuration

```
model SentSmsLog {
  id      String   @id @default(uuid())
  sent_at DateTime @default(now())
}
```

Commit: 11. model: define EmailVerification model for transactional OTPs

```
model EmailVerification {
  email String @id
  code  String
}
```

Commit: 12. model: define Announcement schema for donation campaigns

```
model Announcement {
  id      String @id @default(uuid())
  content String
}
```

Commit: 13. model: define HospitalStock schema with hospital relation keys

```
model HospitalStock {
  id          String @id @default(uuid())
  hospital_id String
}
```

Commit: 14. model: define HospitalInterRequest table model relationships

```
model HospitalInterRequest {
  id           String @id @default(uuid())
  requester_id String
}
```

Commit: 15. model: define AuditLog table model schema config

```
model AuditLog {
  id      String   @id @default(uuid())
  action  String
  details String
}
```

Commit: 16. model: define Appointment model fields mapping to donor fayda

```
model Appointment {
  id       String   @id @default(uuid())
  fayda_id String
  slot     DateTime
}
```

Commit: 17. model: define SmsQueue schema parameters

```
model SmsQueue {
  id      String @id @default(uuid())
  message String
}
```

Commit: 18. model: define Patient hms model schema in main db

```
model Patient {
  id          String @id @default(uuid())
  full_name   String
  ward        String
}
```

Commit: 19. model: define BloodOrder hms model schema in main db

```
model BloodOrder {
  id          String @id @default(uuid())
  blood_type   String
  status       String @default("pending")
}
```

Commit: 20. db: add database index constraints declarations on map tags

```
  @@map("password_reset_requests")
```

DANIEL (Daniel1698-w)

Exclusive File: **`backend/src/controllers/hmsController.js`**
Feature Branch: *`git checkout -b daniel/hms-endpoints`*

Commit: 1. init: import prisma mainDb client configuration

```
const { mainDb } = require('../config/prisma');
```

Commit: 2. feat: add skeleton admitPatient request handler

```
const admitPatient = async (req, res) => {
  // Controller implementation
};
```

Commit: 3. feat: validate incoming patient registration form payloads

```
if (!req.body.full_name || !req.body.ward) {
  return res.status(400).json({ error: 'Missing parameters' });
}
```

Commit: 4. feat: build new Patient record with admitted status in database

```
const patient = await mainDb.patient.create({
  data: { ...req.body, admission_status: 'admitted' }
});
```

Commit: 5. feat: add skeleton getPatients dashboard controller

```
const getPatients = async (req, res) => {
  // Retrieve active list
};
```

Commit: 6. feat: filter getPatients by current hospital owner status query

```
const patients = await mainDb.patient.findMany({
  where: { hospital_id: req.user.id }
});
```

Commit: 7. feat: add getPatientById details search resolver

```
const patient = await mainDb.patient.findUnique({
  where: { id: req.params.id }
});
```

Commit: 8. include: include blood orders history inside patient details payload

```
include: { bloodOrders: true }
```

Commit: 9. feat: add skeleton dischargePatient query handler

```
const dischargePatient = async (req, res) => {
  // Set status
};
```

Commit: 10. feat: update patient admission status to discharged

```
await mainDb.patient.update({
  where: { id: req.params.id },
  data: { admission_status: 'discharged' }
});
```

Commit: 11. feat: add createBloodOrder logic with validation rules

```
const createBloodOrder = async (req, res) => {
  const { patient_id, blood_type, units_needed } = req.body;
};
```

Commit: 12. feat: query hospitalStock record to verify local availability

```
const stock = await mainDb.hospitalStock.findUnique({
  where: { hospital_id_blood_type: { hospital_id, blood_type } }
});
```

Commit: 13. feat: create full local fulfillment branch with decrement update

```
if (stock && stock.quantity >= units_needed) {
  await mainDb.hospitalStock.update({ data: { quantity: { decrement: units_needed } } });
}
```

Commit: 14. feat: create partial local stock decrement and calculate shortfall

```
const shortfall = units_needed - stock.quantity;
await mainDb.hospitalStock.update({ data: { quantity: 0 } });
```

Commit: 15. feat: create hospitalRequest for partial order shortfall

```
await mainDb.hospitalRequest.create({
  data: { blood_type, units_needed: shortfall }
});
```

Commit: 16. feat: create hospitalRequest for full order shortfall

```
await mainDb.hospitalRequest.create({
  data: { blood_type, units_needed }
});
```

Commit: 17. socket: emit requisition_updated websocket payload on full request

```
io.emit('requisition_updated', { status: 'pending', units_needed });
```

Commit: 18. socket: emit requisition_updated websocket payload on partial request

```
io.emit('requisition_updated', { status: 'pending', units_needed: shortfall });
```

Commit: 19. feat: add getBloodOrders list query filtered by hospital ID

```
const orders = await mainDb.bloodOrder.findMany({
  where: { hospital_id: req.user.id }
});
```

Commit: 20. feat: add markTransfused and cancelBloodOrder status managers

```
await mainDb.bloodOrder.update({
  where: { id: req.params.id },
  data: { status: 'transfused' }
});
```

EBISA (Ebisa-Mitiku)

Exclusive File: **`frontend/src/services/api.js`**

Feature Branch: *`git checkout -b ebisa/api-recovery-hooks`*

Commit: 1. init: setup api backend base url configuration

```
const BASE_URL = 'http://localhost:5000/api';
```

Commit: 2. auth: add api.auth.login network fetch request

```
login: async (email, password) => {
  const res = await fetch(`${BASE_URL}/auth/login`, { method: 'POST', body: JSON.stringify({ email,
password }) });
}
```

Commit: 3. auth: add api.auth.registerVerifyEmail validation request

```
registerVerifyEmail: async (email) => {
  return fetch(`${BASE_URL}/auth/register-verify-email`, { method: 'POST', body:
JSON.stringify({ email }) });
}
```

Commit: 4. auth: add api.auth.verifyCode verification query

```
verifyCode: async (email, code) => {
  return fetch(`${BASE_URL}/auth/verify-code`, { method: 'POST', body: JSON.stringify({ email, code }) });
}
```

Commit: 5. auth: add api.auth.registerComplete payload submission

```
registerComplete: async (data) => {
  return fetch(`${BASE_URL}/auth/register-complete`, { method: 'POST', body: JSON.stringify(data) });
}
```

Commit: 6. auth: add api.auth.faydaLookup demographics query

```
faydaLookup: async (faydaId) => {
  return fetch(`${BASE_URL}/auth/fayda-lookup/${faydaId}`);
}
```

Commit: 7. auth: add api.auth.forgotPassword code dispatcher

```
forgotPassword: async (email) => {
  return fetch(`${BASE_URL}/auth/forgot-password`, { method: 'POST', body:
JSON.stringify({ email }) }).then(r => r.json());
}
```

Commit: 8. auth: add api.auth.resetPasswordDonor password updater

```
resetPasswordDonor: async (email, code, newPassword) => {
  return fetch(`${BASE_URL}/auth/reset-password-donor`, { method: 'POST', body: JSON.stringify({ email,
code, newPassword }) });
}
```

Commit: 9. admin: add api.admin.getUsers registry fetcher

```
getUsers: async () => {
  return fetch(`${BASE_URL}/admin/users`, { headers: getHeaders() }).then(r => r.json());
}
```

Commit: 10. admin: add api.admin.updateUserStatus registry status updater

```
updateUserStatus: async (id, status) => {
  return fetch(`${BASE_URL}/admin/users/${id}/status`, { method: 'POST', body:
JSON.stringify({ status }) });
}
```

Commit: 11. admin: add api.admin.getAnalytics report compiler

```
getAnalytics: async () => {
  return fetch(`${BASE_URL}/admin/analytics`, { headers: getHeaders() }).then(r => r.json());
}
```

Commit: 12. admin: add api.admin.triggerReminders reminders initiator

```
triggerReminders: async () => {
  return fetch(`${BASE_URL}/admin/reminders/trigger`, { method: 'POST', headers: getHeaders() });
}
```

Commit: 13. admin: add api.admin.getAuditLogs records query

```
getAuditLogs: async () => {
  return fetch(`${BASE_URL}/admin/audit-logs`, { headers: getHeaders() }).then(r => r.json());
}
```

Commit: 14. admin: add api.admin.getPasswordResetRequests ticket query

```
getPasswordResetRequests: async () => {
  return fetch(`${BASE_URL}/admin/reset-requests`, { headers: getHeaders() }).then(r => r.json());
}
```

Commit: 15. admin: add api.admin.resolvePasswordReset ticket resolution handler

```
resolvePasswordReset: async (id, newPassword) => {
  return fetch(`${BASE_URL}/admin/reset-requests/${id}/resolve`, { method: 'POST', body:
JSON.stringify({ newPassword }), headers: getHeaders() });
}
```

Commit: 16. station: add api.station.lookupDonor and registerDonor services

```
lookupDonor: async (id) => {
  return fetch(`${BASE_URL}/station/donor-lookup/${id}`, { headers: getHeaders() });
}
```

Commit: 17. station: add api.station.createSample and getSamples endpoints

```
createSample: async (sample) => {
  return fetch(`${BASE_URL}/station/samples`, { method: 'POST', body: JSON.stringify(sample) });
}
```

Commit: 18. lab: add api.lab.getPendingSamples and submitTestResult endpoints

```
submitTestResult: async (id, result) => {
  return fetch(`${BASE_URL}/lab/samples/${id}/test`, { method: 'POST', body: JSON.stringify(result) });
}
```

Commit: 19. hms: add api.hms.admitPatient and getPatients hms methods

```
admitPatient: async (patient) => {
  return fetch(`${BASE_URL}/hms/patients`, { method: 'POST', body: JSON.stringify(patient), headers:
getHeaders() }).then(r => r.json());
}
```

Commit: 20. hms: add api.hms.createBloodOrder and markTransfused hms methods

```
createBloodOrder: async (order) => {
  return fetch(`${BASE_URL}/hms/orders`, { method: 'POST', body: JSON.stringify(order), headers:
getHeaders() }).then(r => r.json());
}
```

DESALEGN (DesuTaresa)

Exclusive File: **`frontend/src/pages/auth/Login.jsx`**

Feature Branch: *`git checkout -b desalegn/forgot-password-modal`*

Commit: 1. init: import react and useState hooks in Login.jsx

```
import React, { useState } from 'react';
```

Commit: 2. icon: import logIn and ArrowRight icons from lucide

```
import { LogIn, ArrowRight } from 'lucide-react';
```

Commit: 3. icon: import Key and X icons for recovery modal styling

```
import { LogIn, ArrowRight, Key, X } from 'lucide-react';
```

Commit: 4. ui: render login card structure wrapper layout

```
return (
  <div className="glass-card">
    {/* Form structure */}
  </div>
);
```

Commit: 5. ui: render email input secure text field control

```
<input type="email" value={email} onChange={e => setEmail(e.target.value)} required />
```

Commit: 6. ui: render password input secure text field control

```
<input type="password" value={password} onChange={e => setPassword(e.target.value)} required />
```

Commit: 7. ui: add forgot password link alignment under password input

```
<button type="button" onClick={() => setShowForgotModal(true)}>Forgot password?</button>
```

Commit: 8. state: bind login onSubmit event to authentication api call

```
const handleSubmit = async (e) => {
  e.preventDefault();
  await api.auth.login(email, password);
};
```

Commit: 9. ui: render forgot password overlay dialog box wrapper

```
{showForgotModal && (
  <div className="modal-backdrop">
    {/* Dialog Box */}
  </div>
)}
```

Commit: 10. ui: render account recovery email input layout

```
<input type="email" value={forgotEmail} onChange={e => setForgotEmail(e.target.value)} required />
```

Commit: 11. ui: add request reset submission click event handler

```
<form onSubmit={handleForgotPasswordSubmit}>
```

Commit: 12. ui: display workstation ticket request submission success message

```
{forgotSuccess && <div className="alert alert-success">{forgotSuccess}</div>}
```

Commit: 13. ui: render donor verification OTP code text input field

```
<input type="text" placeholder="123456" value={resetCode} onChange={e => setResetCode(e.target.value)} />
```

Commit: 14. ui: render donor new password secure text input field

```
<input type="password" placeholder="New Password" value={newPassword} onChange={e =>
setNewPassword(e.target.value)} />
```

Commit: 15. ui: render donor confirm password secure text input field

```
<input type="password" placeholder="Confirm Password" value={confirmPassword} onChange={e =>
setConfirmPassword(e.target.value)} />
```

Commit: 16. state: bind recovery inputs to update donor credentials API

```
const handleDonorPasswordResetSubmit = async (e) => {
  e.preventDefault();
  await api.auth.resetPasswordDonor(forgotEmail, resetCode, newPassword);
};
```

Commit: 17. ui: show validation warning on recovery password mismatch

```
if (newPassword !== confirmPassword) {
  setForgotError('Passwords do not match');
  return;
}
```

Commit: 18. ui: display donor password update completion success banner

```
setForgotSuccess('Password updated successfully!');
```

Commit: 19. state: clear forgot password modal states on dialog close

```
setShowForgotModal(false); setForgotEmail(''); setIsDonorReset(false);
```

Commit: 20. ui: style login submit button states and loading spinners

```
<button type="submit">{loading ? 'Signing In...' : 'Sign In'}</button>
```

DESTA (hailedesta60-glitch)

Exclusive File: **`frontend/src/pages/dashboards/HospitalDashboard.jsx`**
Feature Branch: *`git checkout -b desta/hospital-hms-view`*

Commit: 1. init: import package, truck, search and stethoscope icons

```
import { Package, Truck, Search, Stethoscope } from 'lucide-react';
```

Commit: 2. state: add hmsPatients and hmsOrders list state arrays

```
const [hmsPatients, setHmsPatients] = useState([]);
const [hmsOrders, setHmsOrders] = useState([]);
```

Commit: 3. state: add showAdmitForm and showOrderForm boolean states

```
const [showAdmitForm, setShowAdmitForm] = useState(false);
const [showOrderForm, setShowOrderForm] = useState(false);
```

Commit: 4. state: add admitForm input payload values state object

```
const [admitForm, setAdmitForm] = useState({ full_name: '', ward: '' });
```

Commit: 5. state: add orderForm input payload values state object

```
const [orderForm, setOrderForm] = useState({ patient_id: '', blood_type: 'O+' });
```

Commit: 6. state: add orderFulfillmentMsg status notification state

```
const [orderFulfillmentMsg, setOrderFulfillmentMsg] = useState(null);
```

Commit: 7. state: add hmsFilter query selection tab state

```
const [hmsFilter, setHmsFilter] = useState('admitted');
```

Commit: 8. query: add loadHmsData fetch coordinator wrapper

```
const loadHmsData = async () => {
  const patients = await api.hms.getPatients();
  setHmsPatients(patients);
};
```

Commit: 9. query: bind loadHmsData triggers to hms navigation tab click

```
useEffect(() => {
  if (tab === 'hms') loadHmsData();
}, [tab]);
```

Commit: 10. ui: render main Patients & Orders dashboard section container

```
return (
  <div className="hms-container">
    {/* Layout views */}
  </div>
);
```

Commit: 11. ui: render admit patient popup form overlays

```
showAdmitForm && (
  <div className="form-card">
    <h3>Admit New Patient</h3>
  </div>
)
```

Commit: 12. ui: render blood order placement form overlays

```
showOrderForm && (
  <div className="form-card">
    <h3>New Blood Order</h3>
  </div>
)
```

Commit: 13. state: bind patient admission submit to admit API method

```
await api.hms.admitPatient(admitForm);
loadHmsData();
```

Commit: 14. state: bind blood order submit to order creation API method

```
const result = await api.hms.createBloodOrder(orderForm);
setOrderFulfillmentMsg(result._message);
```

Commit: 15. ui: render orderFulfillmentMsg details notification banners

```
{orderFulfillmentMsg && <div className="alert">{orderFulfillmentMsg}</div>}
```

Commit: 16. ui: render admitted/discharged patients list split tab panel

```
hmsPatients.filter(p => p.admission_status === hmsFilter).map(patient => (
  <div key={patient.id}>{patient.full_name}</div>
))
```

Commit: 17. ui: render active blood orders progress lists

```
hmsOrders.map(order => (
  <div key={order.id}>Order for {order.patient_name}</div>
))
```

Commit: 18. ui: add markTransfused click handler triggers to dispatched orders

```
<button onClick={() => api.hms.markTransfused(order.id).then(loadHmsData)}>Mark Transfused</button>
```

Commit: 19. ui: add cancelBloodOrder click handler triggers to pending orders

```
<button onClick={() => api.hms.cancelBloodOrder(order.id).then(loadHmsData)}>Cancel</button>
```

Commit: 20. ui: add responsive layout styling for HMS database rows

```
<div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '20px' }}>
```